

Queue Design

การออกแบบ Queue สำหรับระบบ Emergency Call Dispatch

ระบบ Emergency Call Dispatch ออกแบบ Queue จำนวน 2 รูปแบบ เพื่อเปรียบเทียบประสิทธิภาพและความเหมาะสมในการจัดลำดับเหตุการณ์ฉุกเฉิน ได้แก่ FIFO Event Queue และ Priority Event Queue

1. ข้อมูลที่จัดเก็บใน Queue

ระบบกำหนดข้อมูลของแต่ละเหตุการณ์เป็น Event โดยประกอบด้วยข้อมูลดังนี้

Attribute	Data Type	รายละเอียด
Event ID	String	รหัสประจำเหตุการณ์
Event Type	String	ประเภทของเหตุการณ์
Time	String	เวลาที่รับแจ้งเหตุ
Severity	int	ระดับความรุนแรง
Location	String	สถานที่เกิดเหตุ
Sequence	int	ลำดับการเข้ามาของ Event

ระดับความรุนแรง

Severity	Priority	ความหมาย
Critical	1	เหตุฉุกเฉินร้ายแรงมาก
High	2	เหตุร้ายแรง
Medium	3	เหตุระดับปานกลาง
Low	4	เหตุทั่วไป

หลักการ: ค่า Priority ยิ่งน้อย หมายถึงความสำคัญยิ่งสูง

2. Queue Design แบบที่ 1: FIFO Event Queue

Data Structure

ใช้

```
Queue<Event> queue = new ArrayDeque<>();
```

หรือกำหนดเป็น

```
ArrayDeque<Event> queue = new ArrayDeque<>();
```

หลักการทำงาน

ใช้หลัก **First In First Out (FIFO)** หมายถึง Event ที่เข้ามาก่อนจะได้รับการประมวลผลก่อน โดยไม่พิจารณาระดับความรุนแรง

ตัวอย่าง

ENQUEUE

E1 → E2 → E3 → E4 → E5 → E6

↑

Front

เมื่อเรียก dequeue() จะได้

E1 → E2 → E3 → E4 → E5 → E6

↑

นำออกก่อน

ดังนั้นลำดับการประมวลผลคือ

E1 → E2 → E3 → E4 → E5 → E6

ข้อดี

- เข้าใจง่าย
- Implement ได้ง่าย
- enqueue() และ dequeue() มีประสิทธิภาพ $O(1)$
- มีความยุติธรรมตามลำดับเวลาที่แจ้ง

ข้อจำกัด

- ไม่พิจารณาความรุนแรงของเหตุการณ์
- เหตุการณ์ Critical อาจต้องรอเหตุการณ์ Low ที่เข้ามาก่อน

3. Queue Design แบบที่ 2: Priority Event Queue

Data Structure

ใช้

```
PriorityQueue<Event> queue;
```

โดยกำหนด Comparator เพื่อจัดลำดับตาม Severity

```
PriorityQueue<Event> queue =  
  
    new PriorityQueue<>(  
  
        Comparator.comparingInt(Event::getSeverity)  
  
            .thenComparingInt(Event::getSequence)  
  
    );
```

หลักการทำงาน

Priority Queue จะเลือก Event ที่มี Priority สูงที่สุดออกมาก่อน

กำหนดลำดับ

```
Critical = 1  
  
High     = 2  
  
Medium   = 3  
  
Low      = 4
```

ดังนั้น

```
1 → 2 → 3 → 4
```

หมายถึง

```
Critical → High → Medium → Low
```

ตัวอย่าง

กำหนด Event ดังนี้

E1 = Medium

E2 = Low

E3 = Critical

E4 = High

E5 = Critical

E6 = Medium

เมื่อใส่ทั้งหมดเข้า Priority Queue ระบบจะจัดลำดับตาม Priority เป็น

E3 → E5 → E4 → E1 → E6 → E2

โดย

E3 = Critical = 1

E5 = Critical = 1

E4 = High = 2

E1 = Medium = 3

E6 = Medium = 3

E2 = Low = 4

4. การจัดการกรณี Priority เท่ากัน

กรณีที่ Event มี Priority เท่ากัน ระบบจะใช้ Sequence เป็นตัวตัดสิน

ตัวอย่าง

E3 = Critical = 1

E5 = Critical = 1

เนื่องจาก

E3 เข้ามาก่อน E5

จึงต้องประมวลผล

E3 → E5

ไม่ใช่

E5 → E3

ดังนั้น Comparator จึงมี 2 เงื่อนไข

```
Comparator.comparingInt(Event::getSeverity)  
    .thenComparingInt(Event::getSequence)
```

ความหมายคือ

1. เปรียบเทียบ Severity ก่อน
2. ถ้า Severity เท่ากัน ให้เปรียบเทียบ Sequence

5. Queue Operations

ระบบออกแบบให้รองรับ Operation หลักดังนี้

Operation	ความหมาย
enqueue() / offer()	เพิ่ม Event เข้า Queue
dequeue() / poll()	นำ Event ออกจาก Queue
peek()	ดู Event ที่อยู่ด้านหน้าโดยไม่ลบ
search()	ค้นหา Event
display()	แสดง Event ทั้งหมด
cancel()	ยกเลิก Event ที่อยู่ใน Queue

6. การออกแบบ Cancel Event

ระบบต้องรองรับการยกเลิกเหตุการณ์ที่อยู่ใน Queue

กรณี Event อยู่ใน Queue

ก่อนยกเลิก

[E1, E2, E3, E4, E5]

ยกเลิก E3

หลังยกเลิก

[E1, E2, E4, E5]

กรณี Event ไม่มีอยู่ใน Queue

เช่น ยกเลิก

E99

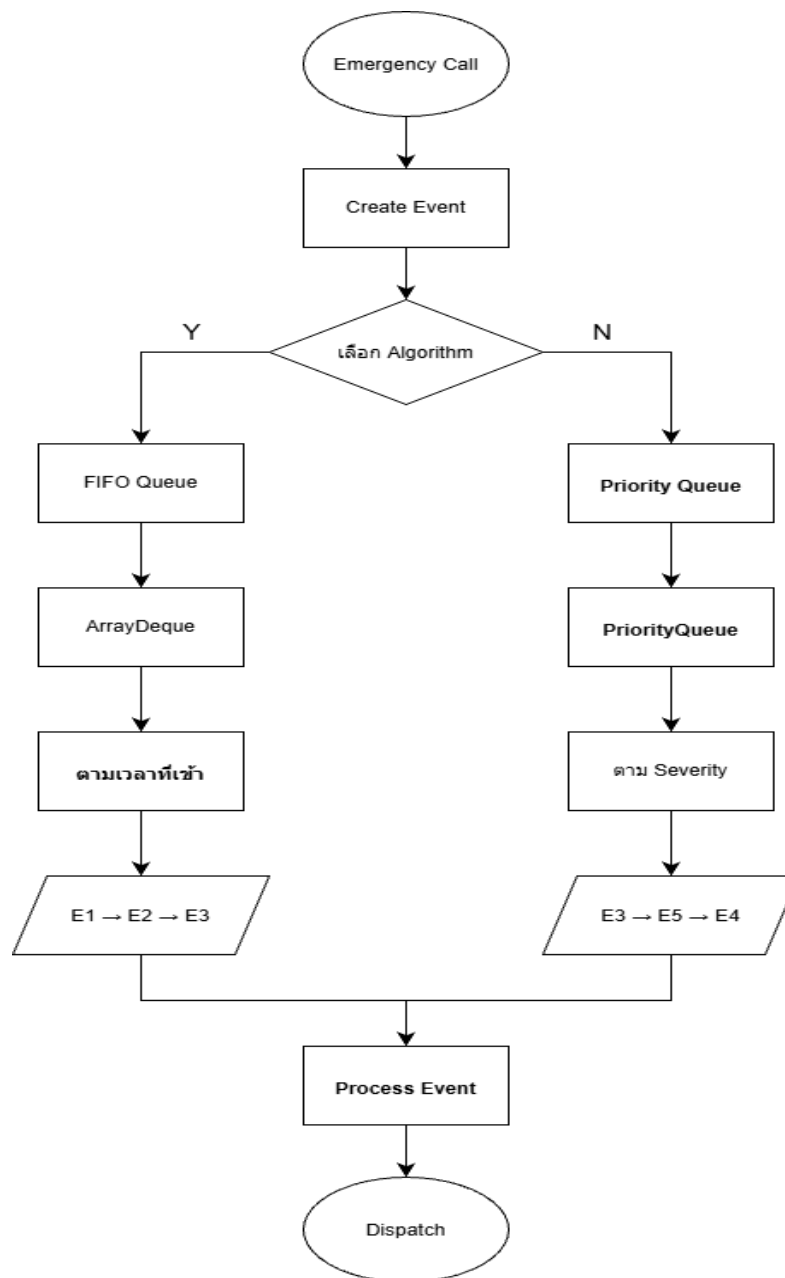
แต่ไม่มี E99 อยู่ใน Queue

ระบบจะแสดง

ไม่พบ Event E99 ใน Queue

7. ภาพรวม Queue Design

สามารถเขียนเป็น Flow ได้ดังนี้



8. เปรียบเทียบ Queue Design

ประเด็น	FIFO Event Queue	Priority Event Queue
Data Structure	ArrayDeque	PriorityQueue
หลักการ	เข้าก่อนออกก่อน	Priority สูงออกก่อน
พิจารณา Severity	ไม่พิจารณา	พิจารณา
พิจารณาเวลา	พิจารณา	ใช้เมื่อ Priority เท่ากัน
Enqueue	$O(1)$	$O(\log n)$
Dequeue	$O(1)$	$O(\log n)$
Peek	$O(1)$	$O(1)$
Search	$O(n)$	$O(n)$
Space	$O(n)$	$O(n)$
ความยุติธรรมตามเวลา	สูง	ปานกลาง
เหมาะกับ	เหตุทั่วไป	เหตุฉุกเฉิน

9. Design Decision

สำหรับระบบ Emergency Call Dispatch ทั้งสองโครงสร้างสามารถใช้งานได้ แต่ Priority Queue เหมาะสมกับระบบจริงมากกว่า เพราะเหตุการณ์ฉุกเฉินไม่ควรได้รับการจัดการเฉพาะตามเวลาที่แจ้ง แต่ควรพิจารณาความรุนแรงร่วมด้วย

ตัวอย่างเช่น หากมี

E1 = Low

E2 = Critical

และ E1 เข้ามาก่อน E2

FIFO

E1 → E2

แต่ Priority Queue

E2 → E1

ซึ่งเหมาะสมกว่าในสถานการณ์ที่ต้องให้ความสำคัญกับชีวิตและความปลอดภัย